

Teach Module 3: Object-Oriented Programming in Java

This note uses the bootcamp document only as a roadmap. The teaching content below is original and explains Module 3 in plain language.

Module 3 Topic

Object-Oriented Programming in Java

Object-oriented programming, or OOP, is a way of writing programs by modeling things as objects. An object combines:

- **Data:** what the thing knows
- **Behavior:** what the thing can do

For example, a `BankAccount` object might know its `balance`, and it might be able to `deposit()` or `withdraw()`.

```
public class BankAccount {
    private String accountNumber;
    private double balance;

    public BankAccount(String accountNumber, double openingBalance) {
        this.accountNumber = accountNumber;
        this.balance = openingBalance;
    }

    public void deposit(double amount) {
        balance += amount;
    }

    public void withdraw(double amount) {
        if (amount <= balance) {
            balance -= amount;
        }
    }

    public double getBalance() {
        return balance;
    }
}
```

A **class** is the blueprint. An **object** is the real thing created from that blueprint.

```
BankAccount account = new BankAccount("A100", 500.00);
account.deposit(100.00);

System.out.println(account.getBalance()); // 600.0
```

Encapsulation

Encapsulation means protecting an object's internal data. Notice that `balance` is `private`. Other code cannot directly do this:

```
account.balance = -999999; // Not allowed
```

Instead, outside code must use methods like `deposit()` and `withdraw()`. This is good because the class controls its own rules.

Constructors

A constructor runs when an object is created. Its job is to set up the object's starting state.

```
public BankAccount(String accountNumber, double openingBalance) {  
    this.accountNumber = accountNumber;  
    this.balance = openingBalance;  
}
```

`this.accountNumber` means "the field that belongs to this object."

Inheritance

Inheritance lets one class reuse and extend another class.

```
public class SavingsAccount extends BankAccount {  
    private double interestRate;  
  
    public SavingsAccount(String accountNumber, double openingBalance, double interestRate)  
    {  
        super(accountNumber, openingBalance);  
        this.interestRate = interestRate;  
    }  
}
```

Here, `SavingsAccount` is a specialized kind of `BankAccount`. The `super(...)` call invokes the parent class constructor.

Polymorphism

Polymorphism means one type can refer to different specific object types.

```
BankAccount account = new SavingsAccount("S200", 1000.00, 0.03);
```

The variable type is `BankAccount`, but the actual object is a `SavingsAccount`. This becomes powerful when different subclasses share a common parent but behave differently.

Abstraction

Abstraction means focusing on what something does, not all the internal details. Interfaces are a common Java tool for abstraction.

```
public interface PaymentProcessor {  
    void processPayment(double amount);  
}
```

Then different classes can implement that behavior:

```
public class CreditCardProcessor implements PaymentProcessor {  
    public void processPayment(double amount) {  
        System.out.println("Processing credit card payment: " + amount);  
    }  
}  
  
public class PaypalProcessor implements PaymentProcessor {  
    public void processPayment(double amount) {  
        System.out.println("Processing PayPal payment: " + amount);  
    }  
}
```

Now your application can depend on the interface:

```
public class CheckoutService {  
    private PaymentProcessor processor;  
  
    public CheckoutService(PaymentProcessor processor) {  
        this.processor = processor;  
    }  
  
    public void checkout(double amount) {  
        processor.processPayment(amount);  
    }  
}
```

This is cleaner because `CheckoutService` does not need to know whether the payment is by card, PayPal, bank transfer, or something else.

OOP Mindset

1. Identify the important things in your problem.
2. Turn those things into classes.
3. Give each class the data it owns.
4. Give each class the behavior it is responsible for.
5. Hide internal details with `private`.
6. Use inheritance only when something truly "is a" specialized version of another thing.
7. Use interfaces when you care about behavior more than implementation.

Practice Exercises

Exercise 1: Student Class

Create a `Student` class with:

- `name`
- `grade`
- constructor
- `updateGrade(double newGrade)`
- `isPassing()`
- `getName()`

Practice goal: classes, fields, constructors, methods, encapsulation.

Exercise 2: Bank Account

Create a `BankAccount` class with:

- `private accountNumber`
- `private balance`
- `deposit(double amount)`
- `withdraw(double amount)`
- `getBalance()`

Add validation so users cannot deposit negative money or withdraw more than the balance.

Practice goal: encapsulation and business rules.

Exercise 3: Employee Payroll

Create an `Employee` class with:

- `name`
- `hourlyRate`
- `hoursWorked`
- `calculatePay()`

Then create a `FullTimeEmployee` and `PartTimeEmployee` class that extend `Employee` .

Practice goal: inheritance.

Exercise 4: Shape Area Calculator

Create a parent class or interface called `Shape` .

Then create:

- `Circle`
- `Rectangle`
- `Triangle`

Each should have a `calculateArea()` method.

Practice goal: abstraction and polymorphism.

Exercise 5: Payment Processor

Create an interface:

```
public interface PaymentProcessor {  
    void processPayment(double amount);  
}
```

```
}
```

Then implement:

- `CreditCardPayment`
- `DebitCardPayment`
- `PaypalPayment`

Create a `CheckoutService` that accepts any `PaymentProcessor` .

Practice goal: interfaces and loose coupling.

Exercise 6: Library System

Create classes:

- `Book`
- `Member`
- `Library`

A member should be able to borrow and return books. A book should know whether it is available.

Practice goal: object relationships and responsibilities.

Exercise 7: Vehicle Inheritance

Create a base class `Vehicle` with:

- `brand`
- `speed`
- `start()`
- `stop()`

Then create:

- `Car`
- `Motorcycle`
- `Truck`

Override a method like `describeVehicle()` in each subclass.

Practice goal: method overriding and inheritance.

Exercise 8: Mini Shopping Cart

Create:

- `Product`
- `CartItem`
- `ShoppingCart`

The cart should support:

- adding products
- removing products
- calculating total price
- printing a receipt

Practice goal: real-world class modeling.

Final Practice Project

Build a small **Banking App** with:

- Customer
- BankAccount
- SavingsAccount
- CheckingAccount
- Transaction
- BankService

Features:

- create customer
- open account
- deposit money
- withdraw money
- transfer between accounts
- print account summary

This ties together classes, encapsulation, constructors, inheritance, polymorphism, and clean design.

Module 3 Lab: Object-Oriented Banking App

Build a small console-based Java app that models bank customers and accounts.

Goal

Practice:

- classes and objects
- constructors
- encapsulation
- inheritance
- polymorphism
- interfaces or abstract classes
- basic SOLID thinking

Part 1: Create Customer

Create a class named `Customer` .

Fields:

```
private String customerId;  
private String name;  
private String email;
```

Add:

- constructor
- getters

- `displayCustomerInfo()` method

Part 2: Create `BankAccount`

Create a class named `BankAccount` .

Fields:

```
private String accountNumber;  
private Customer customer;  
protected double balance;
```

Add:

- constructor
- `deposit(double amount)`
- `withdraw(double amount)`
- `getBalance()`
- `displayAccountInfo()`

Rules:

- deposit amount must be greater than `0`
- withdrawal amount must be greater than `0`
- withdrawal cannot exceed balance

Part 3: Create Account Types

Create two subclasses:

```
SavingsAccount extends BankAccount  
CheckingAccount extends BankAccount
```

For `SavingsAccount` :

- add `interestRate`
- add method `applyInterest()`

For `CheckingAccount` :

- add `overdraftLimit`
- allow withdrawal up to `balance + overdraftLimit`

Part 4: Use Polymorphism

In your `main` method, create a list:

```
List<BankAccount> accounts = new ArrayList<>();
```

Add both savings and checking accounts to the list.

Loop through the list and call:

```
account.displayAccountInfo();
```

Even though the list type is `BankAccount` , it should work with both account types.

Part 5: Test The App

In `Main.java` , do this:

1. Create two customers.
2. Create one savings account.
3. Create one checking account.
4. Deposit money into both.
5. Withdraw money from both.
6. Apply interest to the savings account.
7. Print final account details.

Suggested File Structure

```
src/  
  Customer.java  
  BankAccount.java  
  SavingsAccount.java  
  CheckingAccount.java  
  Main.java
```

Stretch Challenge

Add a `Transaction` class:

```
private String type;  
private double amount;  
private String timestamp;
```

Then store each deposit and withdrawal in a transaction history.

Expected Output Example

```
Customer: Priya Shah  
Account: SAV-1001  
Balance: 1050.0  
  
Customer: Marcus Lee  
Account: CHK-2001  
Balance: -100.0
```

Main success target: by the end, you should be able to explain which class owns which responsibility and why `SavingsAccount` and `CheckingAccount` can both be treated as `BankAccount` .